Name: Atharva Waghmode
Roll No. - 59
Div - D15A

Experiment 04: Classification using RapidMiner (Decision Tree & Naïve Bayes)

Aim

Experiment to explore RapidMiner and implement classification models like Decision Tree and Naïve Bayes. To perform classification using the RapidMiner tool on a Student Result dataset, predicting whether a student passes or fails.

Theory

RapidMiner

RapidMiner is a data analytics platform for preparing datasets, building machine-learning models, and reviewing their performance. Its visual, drag-and-drop workflow supports tasks such as classification, regression, clustering, and text analysis.

Key Features:

- No programming required (GUI-based)
- Supports multiple machine learning algorithms (Decision Tree, Naïve Bayes, SVM, etc.)
- Built-in data visualization and evaluation metrics
- Allows integration with Python and R

Classification Model

A classification model assigns an input record to a predefined category, for example Pass/Fail or spam/not spam.

Types of Classification Models:

- Binary Classification: Two classes (e.g., Pass/Fail, Yes/No)
- Multiclass Classification: More than two classes (e.g., Low/Medium/High)

Common Classification Algorithms:

- Decision Tree: A rule-based model that splits data based on feature values.
- Naïve Bayes: A probabilistic model based on Bayes' Theorem.
- Random Forest: A collection of multiple decision trees.
- Logistic Regression: A statistical model for binary classification.

Naïve Bayes (NB) Classifier

Naive Bayes is a probability-based classifier that applies Bayes' Theorem. It is called naive because it treats the input features as conditionally independent once the class is known.

Bayes' Theorem: $P(Y|X) = [P(X|Y) \times P(Y)] / P(X)$

- P(Y|X) = Probability of class Y given input X
- P(X|Y) = Probability of input X occurring in class Y

- P(Y) = Prior probability of class Y
- P(X) = Probability of input X

Dataset Description

The dataset used is a Student Result Classification dataset, containing academic and demographic details of 500 students, along with their final result (Pass/Fail). It is well suited for classification tasks predicting student outcomes based on study habits, attendance, and other factors. The dataset has 500 rows, each representing a student, and 10 columns, including:

- student_id: Unique ID (numeric, dropped for modeling)
- age: Age of the student in years
- class: Grade/standard the student is in (9th, 10th, 11th, 12th)
- section: Section within the class (A, B, C)
- study_hours_per_week: Average number of hours studied per week
- attendance_pct: Attendance percentage
- subject: Subject the record pertains to (English, Science, History, Computer, Maths)
- city: City the student resides in (Chennai, Delhi, Pune, Bangalore, Mumbai)
- marks_obtained: Marks scored by the student
- Result_Category: Final result (Pass / Fail) — the binary target/label

Implementation Steps (RapidMiner)

- Step 1 – Import the Dataset: Used 'Read CSV' → Import Configuration Wizard to load the student result CSV file, then saved it to the Repository as student_result_classification.
- Step 2 – Preprocess Data: 'Select Attributes' to drop the non-predictive student_id column and keep the remaining 9 attributes; 'Set Role' to assign Result_Category as the label.
- Step 3 – Split the Data: 'Split Data' operator with an 80% / 20% train/test partition ratio.
- Step 4 – Train the Models: 'Decision Tree' (connected to the 80% training partition) and 'Naïve Bayes' (default settings), both trained on the same training partition.
- Step 5 – Apply the Models on Test Data: Separate 'Apply Model' operators for each model, connected to the 20% test partition.
- Step 6 – Evaluate Model Performance: Separate 'Performance (Classification)' operators for each model, producing accuracy and a confusion matrix.
- Step 7 – Compare Naïve Bayes & Decision Tree: Compared accuracy, confusion matrices, and class-wise precision/recall between the two models.

Process Design

The complete RapidMiner process: Retrieve student_result_classification → Select Attributes (drop student_id) → Set Role (Result_Category → label) → Split Data (0.8 / 0.2), feeding the 80% training partition into the Decision Tree operator (and, in parallel, into a Naïve Bayes operator), each followed by its own Apply Model and Performance operator on the 20% test partition.

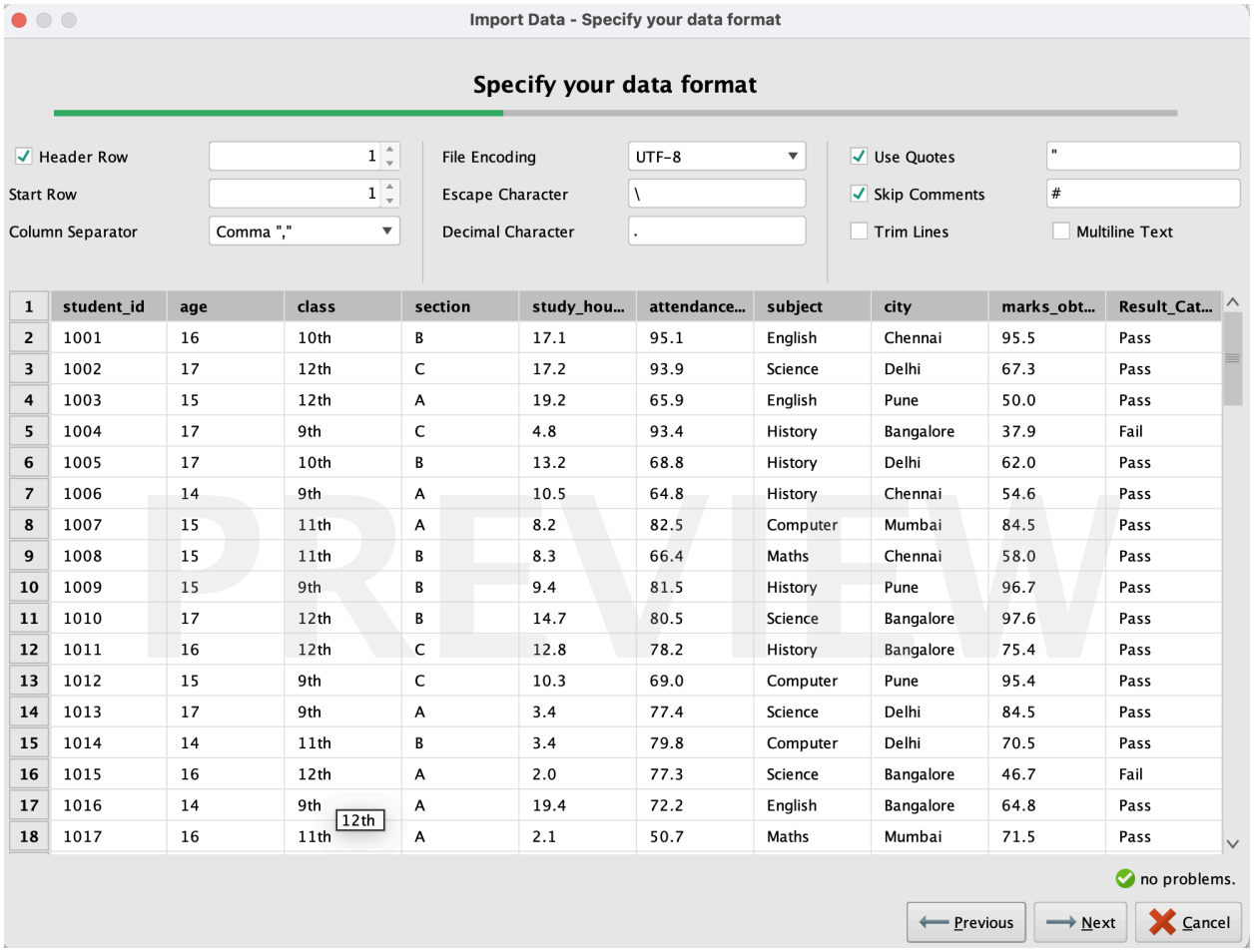


Fig. 1 – Import Configuration Wizard: specifying data format for the student result CSV file.

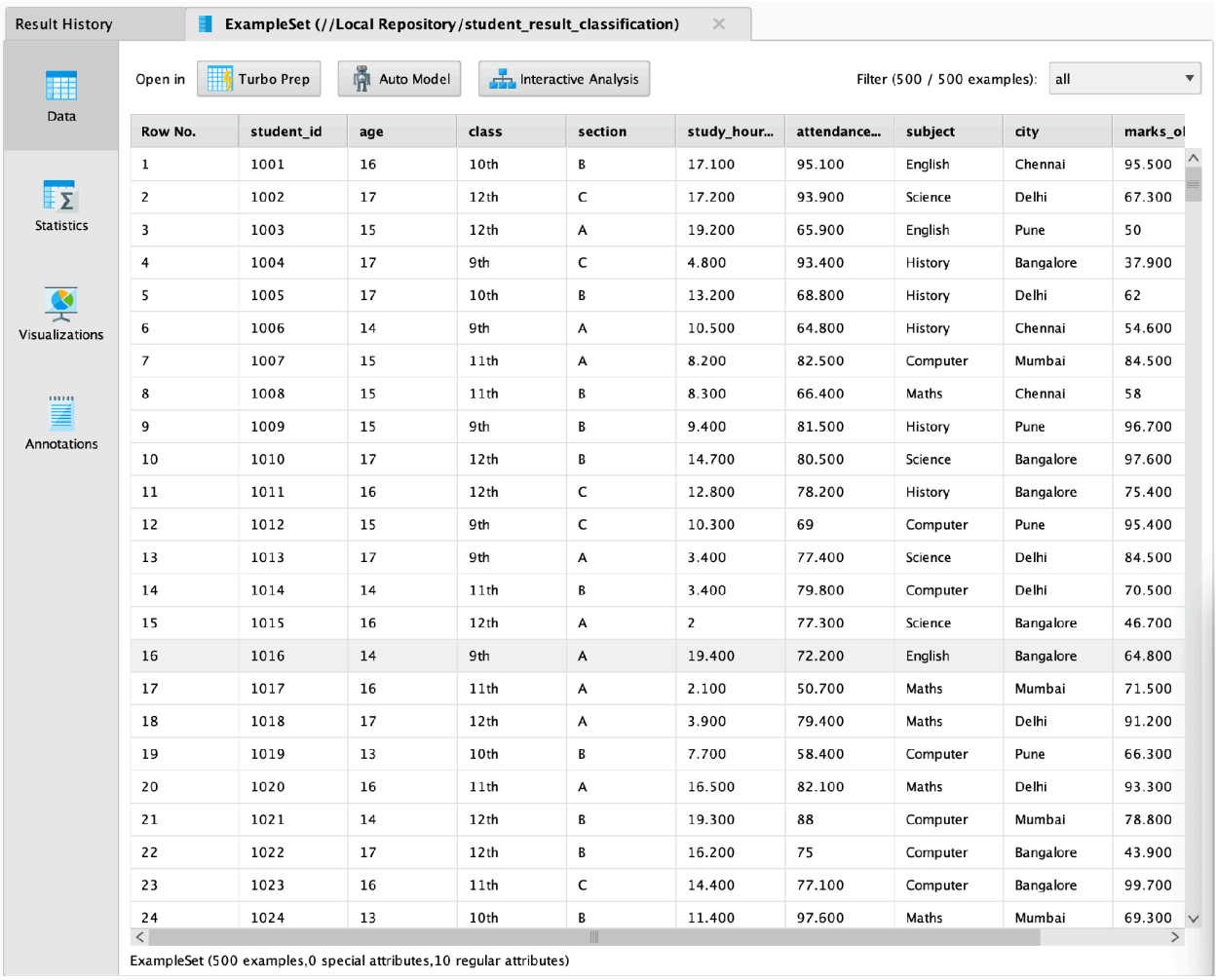


Fig. 2 – Retrieved ExampleSet: student_result_classification (500 examples, 10 attributes).

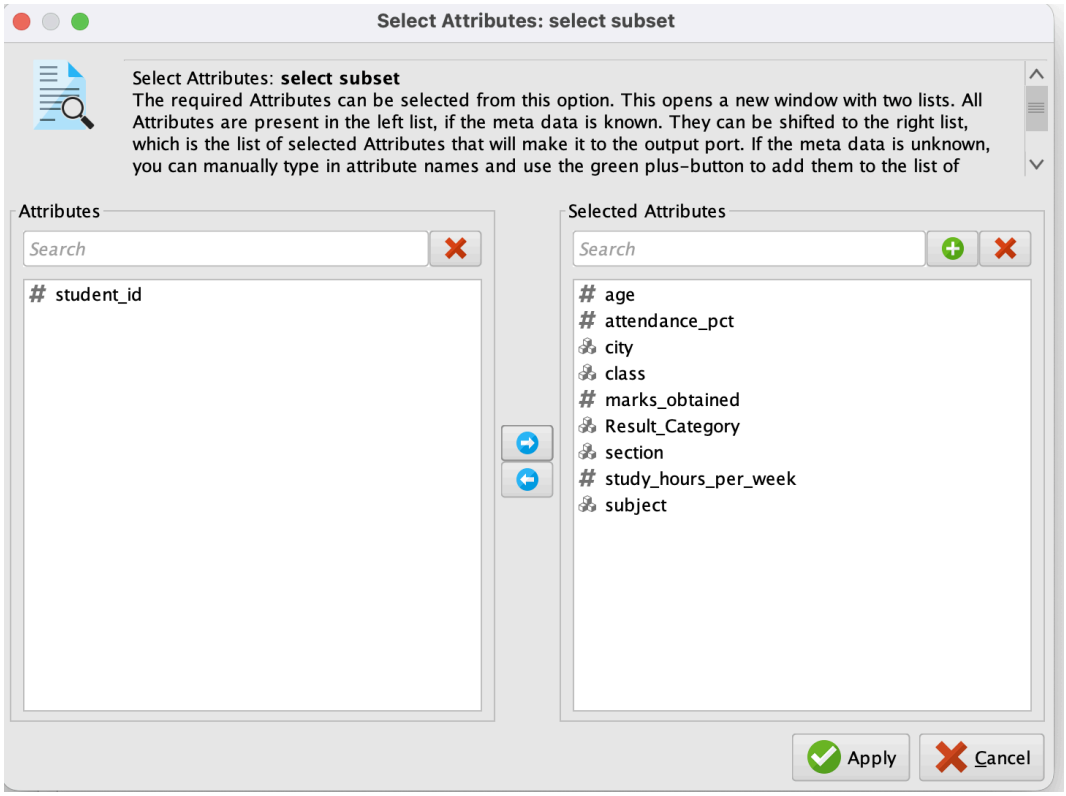


Fig. 3 – Select Attributes operator: keeping all columns except student_id.

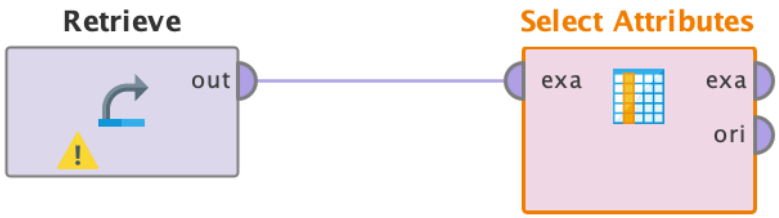


Fig. 4 – Process canvas: Retrieve → Select Attributes.

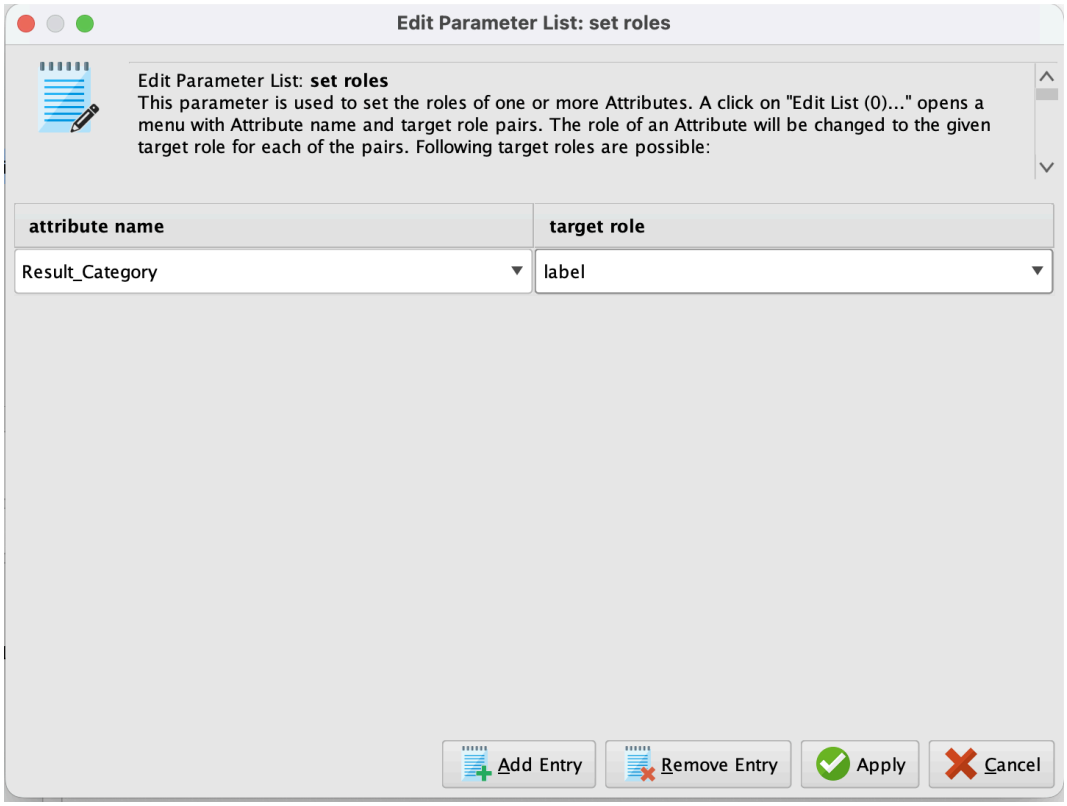


Fig. 5 – Set Role operator: assigning Result_Category as the label.

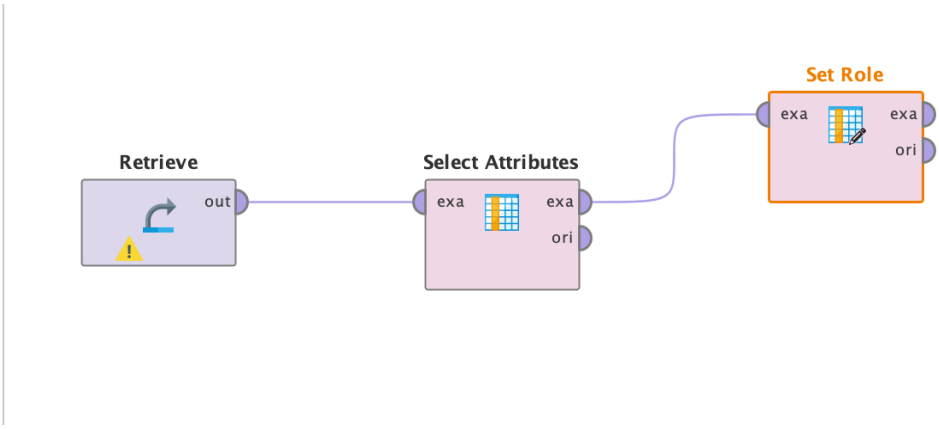


Fig. 6 – Process canvas: Retrieve → Select Attributes → Set Role.

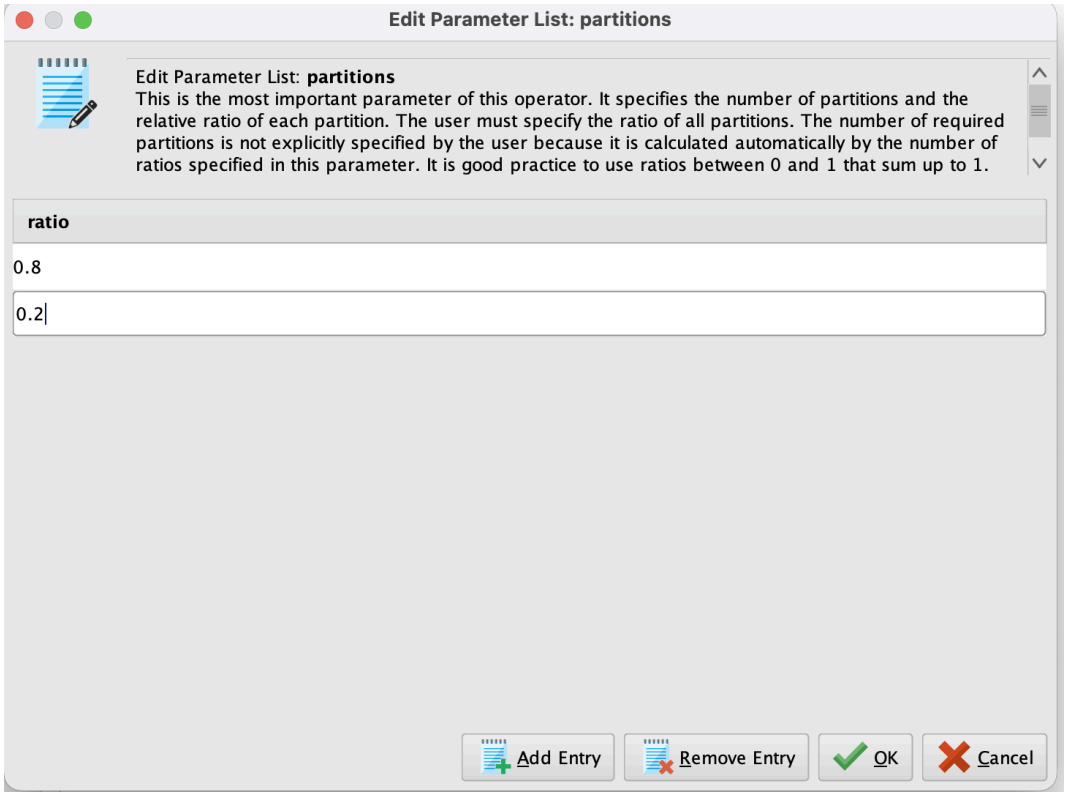


Fig. 7 – Split Data operator: 80% / 20% partition ratio.

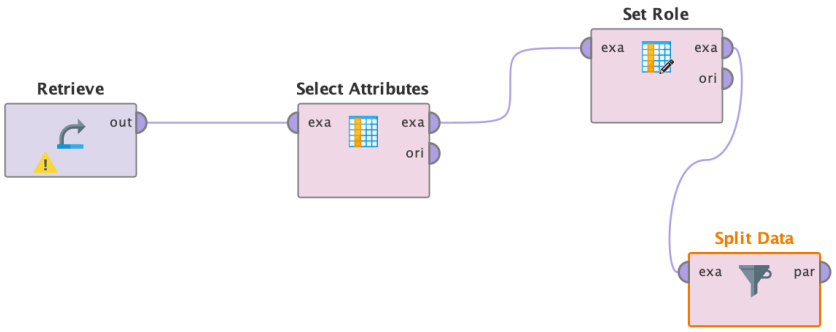


Fig. 8 – Process canvas: Retrieve → Select Attributes → Set Role → Split Data.

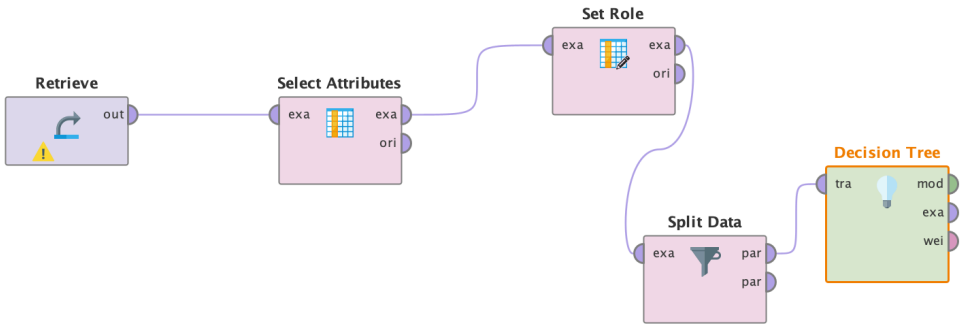


Fig. 9 – Process canvas: full pipeline with the Decision Tree operator connected to the training partition.

Decision Tree – Results

Tree Structure

The first split occurs on study_hours_per_week (threshold ≈ 13.95 hrs/week). For students with lower study hours, attendance_pct is used next; for those with higher study hours, city and section drive further splits, and deeper levels bring in class, subject, and age. Overall, study_hours_per_week and attendance_pct emerge as the strongest predictors of Result_Category, with the model repeatedly using them near the top of the tree.

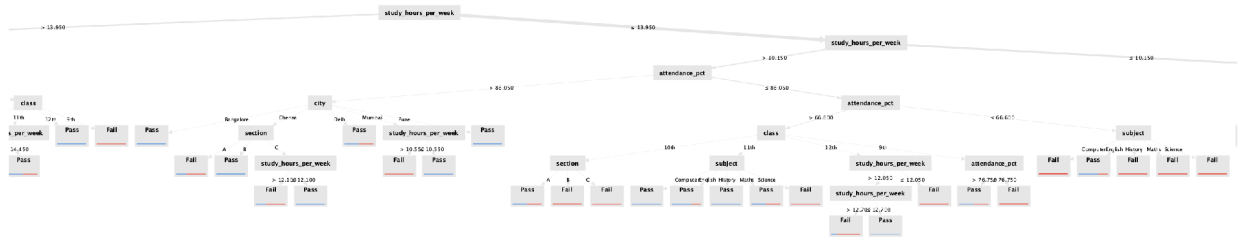


Fig. 10 – Decision Tree (part 1): root split on study_hours_per_week, branching into attendance_pct, city and class.

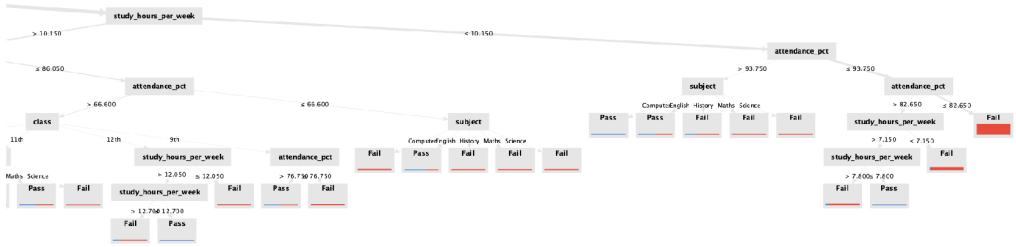


Fig. 11 – Decision Tree (part 2): right-hand branch, further splits on attendance_pct and subject.

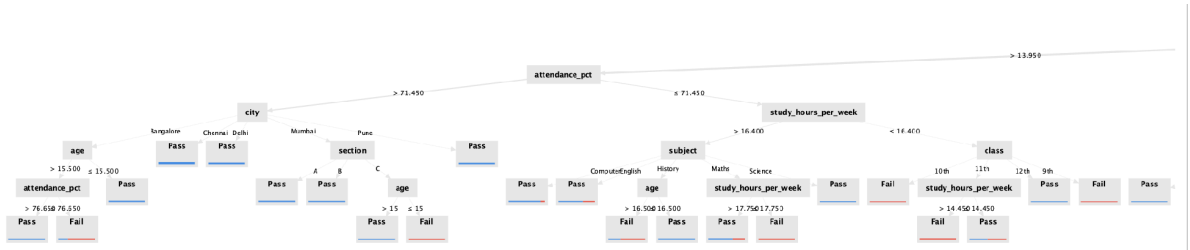
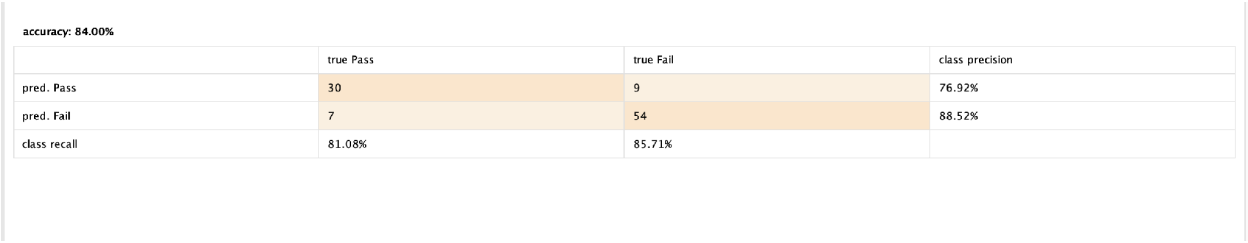


Fig. 12 – Decision Tree (part 3): leftmost branch, splits on attendance_pct, city, section and age.

Performance Vector


accuracy: 84.00%

	true Pass	true Fail	class precision
pred. Pass	30	9	76.92%
pred. Fail	7	54	88.52%
class recall	81.08%	85.71%	

Fig. 13 – Decision Tree Performance Vector.

accuracy: 84.00%

	true Pass	**true Fail**	**class precision**
pred. Pass	30	9	76.92%
pred. Fail	7	54	88.52%
class recall	81.08%	85.71%	

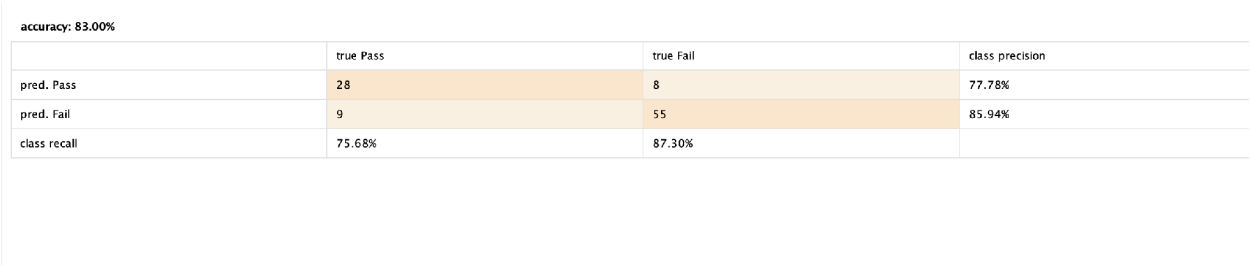
Inferences (Decision Tree):

- Accuracy: 84.00% of predictions on the test set were correct.
- Class "Pass" precision: 76.92%; class "Fail" precision: 88.52% — the model is noticeably more precise when predicting failure.
- Class recall: 81.08% for Pass, 85.71% for Fail — the model catches slightly more actual fails than actual passes, with only 7 false negatives and 9 false positives out of 100 test instances.

Insight: study_hours_per_week is the most influential attribute in classification, followed closely by attendance_pct — together they drive the top three levels of the tree, matching the intuitive expectation that study time and attendance are the strongest determinants of a student's result.

Naïve Bayes – Results

Performance Vector


accuracy: 83.00%

	true Pass	true Fail	class precision
pred. Pass	28	8	77.78%
pred. Fail	9	55	85.94%
class recall	75.68%	87.30%	

Fig. 14 – Naïve Bayes Performance Vector.

accuracy: 83.00%

	true Pass	**true Fail**	**class precision**
pred. Pass	28	8	77.78%

pred. Fail	9	55	85.94%
class recall	75.68%	87.30%	

Inferences (Naïve Bayes):

- Accuracy: 83.00% of predictions were correct — very close to the Decision Tree's accuracy.
- Class "Pass" precision: 77.78%, meaning most predicted "Pass" instances were actually passes.
- Class "Fail" precision: 85.94%, meaning predicted "Fail" instances were highly reliable.
- Class recall: 75.68% for Pass, 87.30% for Fail — similar pattern to the Decision Tree, with the model slightly better at catching actual fails than actual passes.

Comparison

- Decision Tree (84.00%) has marginally higher accuracy than Naïve Bayes (83.00%) on this test split — a difference of only one percentage point.
- Both models recall the "Fail" class better than the "Pass" class (Decision Tree: 85.71% vs 81.08%; Naïve Bayes: 87.30% vs 75.68%), suggesting the models are slightly more reliable at identifying students who will fail.
- Decision Tree provides better interpretability, explicitly showing that study_hours_per_week and attendance_pct, followed by city, class, section, subject and age, drive the classification.
- Naïve Bayes is simpler and faster to train (no tuning needed) but its independence assumption between features costs it a small amount of accuracy and Pass-class recall here.

Conclusion

Both Decision Tree and Naïve Bayes classifiers were successfully implemented in RapidMiner on the Student Result dataset using an 80/20 train-test split. The Decision Tree achieved 84.00% accuracy versus 83.00% for Naïve Bayes, with study_hours_per_week identified as the most influential predictor of a student's result, followed by attendance_pct, city, class, section, subject and age. If interpretability and feature importance matter, Decision Tree is preferable; if speed and simplicity are the priority, Naïve Bayes is a reasonable alternative given its accuracy is nearly identical here. Ensemble methods like Random Forest or boosting techniques (e.g., XGBoost) could potentially improve results further.